Logics & Symbolic AI – Short note

*Ghi chú ngắn bằng tiếng Việt; thuật ngữ **in đậm** giữ nguyên tiếng Anh như trong slide và đề thi.*

Cách dùng. Mỗi phần: *ý tưởng* (hiểu bằng trực giác) → *định nghĩa* (ký hiệu slide) → *cách làm bài* → *lỗi hay gặp*. Lời giải đầy đủ của đề mẫu nằm trong cheat sheet.

1. Propositional logic

Ý tưởng: Mỗi biến mệnh đề chỉ đúng hoặc sai. **Syntax** là cách viết công thức, **semantics** là ý nghĩa (đúng/sai) của nó.

- **Valuation** v: gán True/False cho từng ký hiệu, rồi tính lên theo phép nối: $v(\neg F) = \text{Not } v(F)$, $v((A \bullet B)) = v(A) \blacksquare v(B)$.
- **Tautology**: đúng với *mọi* v (vd. $p \vee \neg p$). **Satisfiable**: đúng với *ít nhất một* v.
- **Logical consequence** $S \models X$: hễ S đúng thì X đúng.
- Định lý quan trọng nhất: $S \models X \iff S \cup \{\neg X\}$ *không thỏa được*. Đây là cơ sở của mọi chứng minh bằng phản chứng / resolution.

Đưa về CNF và resolution

- **CNF** = hội của các **clause**; clause = tuyển các literal. Slide viết $\langle[\ldots],[\ldots]\rangle$: ngoặc vuông = OR, ngoặc nhọn = AND.
- Cách làm: bỏ $\supset$ ($X \supset Y \equiv \neg X \vee Y$), đẩy $\neg$ vào trong (De Morgan), rồi phân phối $\vee$ qua $\wedge$.
- **Resolution**: từ $[A, X]$ và $[\neg X, B]$ suy ra $[A, B]$. Muốn chứng minh X: thêm $[\neg X]$ và hợp giải tới khi được clause rỗng $[]$ (mâu thuẫn).
- **Sound**: chứng minh được thì đúng. **Complete**: đúng thì chứng minh được. Logic mệnh đề: sound, complete, **decidable**.

Dễ sai: phủ định của $X \supset Y$ là $X \wedge \neg Y$, *không* phải $\neg X \supset \neg Y$.

2. First-order logic (FOL)

Ý tưởng: Thêm *đối tượng*, *hàm* và *lượng từ* $\forall, \exists$ để nói "mọi người", "có ai đó".

- **Term** chỉ một đối tượng (biến, hằng, $f(t_1, ..)$); **atomic formula** $p(t_1, ..)$ là một mệnh đề đúng/sai.
- **Function hay predicate?** Ký hiệu nằm *bên trong đối số* của ký hiệu khác ⇒ **function**. Ký hiệu đứng trực tiếp dưới $\neg, \wedge, \vee, \rightarrow$ hoặc lượng từ ⇒ **predicate**. **Arity** = số đối số.
- Biến **free**: không nằm trong phạm vi của lượng từ nào buộc nó. Phạm vi của $\forall x$ chỉ là công thức ngay sau nó.
- **Interpretation**: miền D, hằng ↦ phần tử, hàm ↦ hàm trên D, vị từ ↦ quan hệ. **Valid** = đúng trong mọi mô hình.

Chứng minh F valid bằng resolution (6 bước)

1. Lấy $\neg F$, đưa về **prenex** (mọi lượng từ ra đầu; đổi tên biến trước).
2. **Skolemize**: thay mỗi $\exists x$ bằng hàm mới của các biến $\forall$ đứng trước ($f(x_1..)$, hoặc hằng nếu không có).
3. Bỏ $\forall$. 4. Đưa về CNF.
5. Resolution + **unification** (chọn giá trị cho biến để hai literal khớp).
6. Ra $[] \Rightarrow \neg F$ không thỏa được ⇒ F valid.

Dễ sai: khi kéo lượng từ ra khỏi *về trái* của $\supset$, nó *đổi chiều*: $((\forall v)A \supset B) \equiv (\exists v)(A \supset B)$.

- FOL là **semi-decidable**: nếu F valid thì sẽ tìm ra chứng minh; nếu không, thuật toán có thể chạy mãi.

Natural deduction

- Mỗi phép nối có luật **introduction** (I: tạo ra nó) và **elimination** (E: dùng nó). Ví dụ →I: giả sử p chứng minh được q thì có $p \rightarrow q$; →E = modus ponens.
- Handout: g = gauche (trái), d = droite (phải); Abs = phản chứng.

3. Planning (STRIPS và SAT)

Ý tưởng: Mô tả thế giới bằng các sự kiện (**fluents**); mỗi **action** có **preconditions** và **effects**; tìm dãy hành động đi từ trạng thái đầu tới đích.

- **STRIPS**: tìm kiếm quay lui; **Sussman anomaly**: giải từng đích riêng sẽ phá đích kia.
- **Planning as SAT** (Kautz & Selman): thêm thời gian t vào mọi vị từ, **ground** thành biến boolean (vd. `posX-1-3` = "X ở vị trí 1 lúc $t = 3$"), rồi viết 5 loại ràng buộc:
  1. **Initial state** ($t = 0$): ghi *mọi* fluent, đúng hoặc $\neg$.
  2. **Goal** ($t = N$): chỉ ghi các sự kiện đích.
  3. **Problem constraints**: vd. mỗi bước đúng một hành động.
  4. **Action**: $a_t \Rightarrow$ (precondition lúc t) $\wedge$ (effect lúc $t+1$).
  5. **Explanatory frame axioms**: nếu fluent thay đổi thì phải có hành động gây ra nó. Mỗi fluent 2 axiom: bật lên ($\neg f_t \wedge f_{t+1} \Rightarrow \bigvee$ hành động thêm f) và tắt đi ($f_t \wedge \neg f_{t+1} \Rightarrow \bigvee$ hành động xóa f).
- Không biết số bước N: thử $N = 1, 2, \ldots$ tới khi SAT solver tìm ra mô hình.

Dễ sai: trạng thái đầu phải ghi cả các fluent *sai*; đích thì không. Hành động có precondition không thể đúng (vd. Up khi $Y = 3$) ⇒ viết $\neg U3_t$.

4. Formal Concept Analysis (FCA)

Ý tưởng: Một bảng đối tượng × thuộc tính. Một **concept** là một cặp (nhóm đối tượng, nhóm thuộc tính) "khớp nhau hoàn toàn": các đối tượng có chung đúng các thuộc tính đó, và chỉ chúng có đủ các thuộc tính đó.

- **Context** $K = (X, Y, I)$: $(x, y) \in I$ nghĩa là đối tượng x có thuộc tính y.
- $A^{\uparrow}$ = các thuộc tính *mọi* đối tượng trong A đều có; $B^{\downarrow}$ = các đối tượng có *đủ* mọi thuộc tính trong B.
- **Concept** (A, B): cần **cả hai** $A^{\uparrow} = B$ và $B^{\downarrow} = A$. A = **extent**, B = **intent**.
- Mẹo kiểm tra: tính $B^{\downarrow}$, rồi $(B^{\downarrow})^{\uparrow}$. Nếu được lại đúng B thì B là intent.
- Thứ tự: concept dưới có *ít* đối tượng, *nhiều* thuộc tính. Tập mọi concept là một **complete lattice**.

Intersection method theo hàng (Wille; Stumme)

Coi mỗi hàng là một **transaction** (tập thuộc tính của đối tượng). *Mọi intent là giao của một số hàng*. Cách làm:

1. Ghi intent của từng hàng $\{x\}^{\uparrow}$.
2. Giao mỗi hàng với mọi hàng phía trên; kết quả mới thì thêm thành hàng mới; lặp cho tới hàng mới cuối cùng.
3. Thêm hàng toàn bộ thuộc tính Y nếu chưa có (concept đáy, extent có thể rỗng).
4. Extent của mỗi intent = các đối tượng có đủ intent đó.
5. Vẽ: intent nhỏ ở trên; nối mỗi concept với concept ngay trên nó (intent chứa nó, không qua trung gian).
6. Gán nhãn rút gọn: thuộc tính ở concept *cao nhất* có nó; đối tượng ở concept *thấp nhất* có nó.

- **Implication** $Z_1 \Rightarrow Z_2$ đúng khi mọi đối tượng có Z_1 cũng có Z_2, tức $Z_1^{\downarrow} \subseteq Z_2^{\downarrow}$. Sai thì chỉ ra một *phản ví dụ*.
- **Clarified**: không có hàng (cột) trùng nhau. **Reduced**: không có hàng (cột) là giao của hàng (cột) khác. Cả hai không đổi hình dạng lattice.
- **Duquenne–Guigues basis**: tập implication nhỏ nhất suy ra được mọi implication đúng.

Dễ sai: quên rằng cột đầy (vd. y_1 mọi người đều có) nằm trong *mọi* intent; quên concept đáy $(\emptyset, Y)$.

5. Association rules

Ý tưởng: Trong dữ liệu giỏ hàng, tìm luật "ai mua X thường mua Y". Không phải nhân quả.

- **Support** $s(X)$ = tỉ lệ transaction chứa X. **Confidence** $c(X \Rightarrow Y) = \sigma(X \cup Y)/\sigma(X)$ = trong số chứa X, bao nhiêu chứa cả Y.
- **Lift** $= c/s(Y)$: $= 1$ độc lập, > 1 có liên hệ dương. Confidence cao chưa đủ: Tea⇒Coffee $c = .75$ nhưng $s(Coffee) = .8$ nên thật ra là liên hệ âm.
- **A priori principle**: tập con của tập frequent là frequent ⇒ loại sớm mọi tập chứa một tập con không frequent.
- **Apriori**: tìm L_1; ghép L_{k-1} thành ứng viên C_k, cắt tỉa, quét dữ liệu, giữ cái $\geq minSup$; lặp. Luật từ tập L: $A \Rightarrow L \setminus A$.
- **Liên hệ với FCA**: implication = luật có $c = 1$; intents = **closed itemsets** (cùng support với bao đóng của chúng). FCA hợp dữ liệu nhỏ, sạch; AR hợp dữ liệu lớn, nhiễu.

6. Fuzzy logic

Ý tưởng: Sự thật có *mức độ*: "nóng" đúng 0.9. **Degree of truth** khác xác suất: xác suất nói về *khả năng*, độ đúng nói về *mức độ*.

- NOT: $1-x$. AND = **T-norm**, cần 4 tính chất: giao hoán, đơn điệu, kết hợp, $T(a,1) = a$. OR = **T-conorm** $\perp(a,b) = 1 - T(1-a, 1-b)$.
- Bốn T-norm của slide (từ lớn tới nhỏ): **min** $\min(a,b)$; **product** ab; **Łukasiewicz** $\max(0, a+b-1)$; **drastic**: nếu một bên bằng 1 thì lấy bên kia, ngược lại 0.
- Kiểm tra T-norm: thử $T(a,1) = a$ trước (rất nhanh bác bỏ, vd. trung bình $\frac{a+b}{2}$: $T(.4,1) = .7$).
- **R-implication** $R_T(x,y)$ = giá trị z lớn nhất sao cho $T(z,x) \leq y$. Nếu $x \leq y$ thì luôn $= 1$. Với min: $= y$ khi $x > y$; product: y/x; Łukasiewicz: $1 - x + y$.
- **S-implication** = tính $\neg x \vee y$ trực tiếp.
- **Fuzzy inference**: luật "$C \geq \phi$(tiền đề)". Bắt đầu từ giá trị cho trước (không cho = 0), lặp: nếu luật chưa thỏa thì *nâng* kết luận lên đúng ϕ(tiền đề). Dừng khi không đổi: đó là nghiệm nhỏ nhất.
- **Fuzzy set** $\mu(x) \in [0,1]$; **Mamdani**: fuzzify → firing strength α → cắt tập kết luận ở α → **defuzzify** (First/Mean of Maxima, Median).

Dễ sai: trong inference chỉ được *tăng* giá trị, không giảm.

7. MAX SAT & PSL

Ý tưởng: Luật có *trọng số*, có thể mâu thuẫn nhau; ta muốn thỏa "càng nhiều trọng số càng tốt".

- **Weighted MAX SAT**: biến 0/1; tối đa tổng trọng số clause thỏa. **NP-hard**: cho mọi clause trọng số 1, bài SAT thỏa được $\iff$ tối ưu $= m$; nên giải nhanh MAX SAT thì giải nhanh SAT.
- **PSL**: cho biến nhận giá trị trong $[0,1]$; clause tính bằng OR Łukasiewicz $\min(\sum \text{literal}, 1)$, literal phủ định $= 1 - y$. Mục tiêu $\sum_c w(c) \min(\ldots, 1)$.
- PSL **polynomial**: mỗi hạng tử là min của hàm tuyến tính và 1 ⇒ lõm; bài toán **convex**, viết được thành quy hoạch tuyến tính. Slide giải bằng **hit and run**.
- Làm bài tối ưu nhỏ: chia trường hợp theo $\min(\cdot, 1)$ (khi tổng ≤ 1 và ≥ 1), mỗi miền là tuyến tính nên tối ưu ở đỉnh.

8. Theorem proving & λ-calculus

- **Hilbert's program**: muốn toán học nhất quán, đầy đủ, quyết định được. **Gödel**: hệ nhất quán chứa số học thì không đầy đủ, và không tự chứng minh được tính nhất quán.
- λ-term: biến, $\lambda x.M$ (hàm), MN (áp dụng). **β-reduction**: $(\lambda x.M)N \to M[N/x]$ (thay x bằng N; đổi tên biến buộc nếu sẽ "bắt" biến tự do của N).
- **Church numerals**: $\overline{n} = \lambda f.\lambda x.f^n x$ (áp dụng f n lần). Khi làm bài, *thay số cụ thể* $\overline{m}$ vào chỗ biến m rồi rút gọn.
- True $= \lambda a.\lambda b.a$ (chọn cái đầu), False $= \lambda a.\lambda b.b$ (chọn cái sau).
- **Simply typed** λ: mọi term có kiểu đều **strongly normalizing** (rút gọn luôn dừng); $(\lambda x.xx)(\lambda x.xx)$ không có kiểu.
- **Curry–Howard**: công thức = kiểu, chứng minh = chương trình; →I = λ, →E = áp dụng hàm.

9. Description logics (DL)

Ý tưởng: Một ngôn ngữ để định nghĩa **concepts** (lớp: Person, Mother) từ **roles** (quan hệ: hasChild) mà không cần biến; là mảnh quyết định được của FOL, là nền của OWL.

- **TBox** = định nghĩa chung (Mother $\equiv$ Woman $\sqcap \exists$hasChild.Person). **ABox** = sự kiện về cá thể (Woman(Alice), hasChild(Alice,Bob)).
- **Interpretation** $\mathbb{I} = (\Delta^I, \cdot^I)$: concept $\mapsto$ tập con của Δ^I, role $\mapsto$ tập cặp. $\sqcap$ = giao, $\sqcup$ = hợp, $\neg$ = phần bù.
- $\exists R.C$ = "có *ít nhất một* R-successor thuộc C". $\forall R.C$ = "*mọi* R-successor (nếu có) thuộc C" (đúng cả khi không có successor nào!).
- Họ ngôn ngữ: **AL** (phủ định chỉ cho concept nguyên tử, $\exists R.\top$) + U ($\sqcup$) + E ($\exists R.C$) + C (phủ định đầy đủ) + N (đếm số).
- Dịch sang FOL: $\exists R.C \mapsto \exists y(R(x,y) \wedge C(y))$; $\forall R.C \mapsto$

$\forall y\,(R(x,y) \to C(y))$; $C \sqsubseteq D \mapsto \forall x\,(C(x) \to D(x))$.

Suy luận

- TBox: **satisfiability**, **subsumption** $C \sqsubseteq D$, **equivalence**, **disjointness**. Mọi bài quy về *unsatisfiability*: $C \sqsubseteq D \iff C \sqcap \neg D$ không thỏa được.
- ABox: **consistency**, **instance check** ($A \models C(a) \iff A \cup \{\neg C(a)\}$ mâu thuẫn), **retrieval**, **realization**.
- Chứng minh $C \sqsubseteq D$ theo ngữ nghĩa: "lấy $x \in C^I$ tùy ý, chứng minh $x \in D^I$".
- Kiểm tra cá thể (vd. Pedro là SuccessfulMan?): xét từng thành phần của định nghĩa; $\exists$ cần một *nhân chứng* trong ABox; cần định nghĩa là $\equiv$ (đủ), $\sqsubseteq$ thì không suy ra được.

Dễ sai: phản ví dụ cho $\exists r.A \sqcap \exists r.B \sqsubseteq \exists r.(A \sqcap B)$: x có một con trong A và một con *khác* trong B.

OWL, ontology, knowledge base

- **OWL 2 DL** = SROIQ(D); công cụ Protégé, reasoner HermiT/Pellet. **EquivalentTo** = điều kiện cần & đủ; **SubClassOf** = chỉ cần.
- Property: **transitive**, **symmetric**, **functional** (tối đa một giá trị), **inverse functional**, **inverse**.
- **Ontology** = "formal, explicit specification of a shared conceptualization" (4 chữ: formal = máy hiểu, explicit = định nghĩa rõ, shared = đồng thuận, conceptualization = cách nhìn lĩnh vực). Cấu trúc $O = \{\mathscr{C}, \mathscr{R}, \mathscr{H}^{\mathscr{C}}, Rel, \mathscr{A}^O\}$.
- **Knowledge base** $KB = \{O, I, inst, instr\}$: ontology + cá thể + gán cá thể cho concept / relation.
- **Forward chaining**: từ sự kiện, áp luật tới khi ra đích. **Backward chaining**: từ đích, tìm luật kết luận ra nó, chứng minh tiền đề.

10. Chiến lược làm bài

- Đọc kỹ động từ của đề: *give, prove, justify, explain*. "Justify" = phải có một câu lý do.
- Định nghĩa có nhiều điều kiện (concept, T-norm, Galois): kiểm tra *đủ số* điều kiện.
- FCA: luôn kiểm tra lại bằng $(\cdot)^{\downarrow\uparrow}$; đếm số concept cuối cùng.
- Fuzzy/PSL: ghi rõ T-norm dùng, tính từng bước.
- Chứng minh NP-hard: "quy một bài NP-complete đã biết về bài này trong thời gian đa thức".